

Table of Contents

✓ Resolução — Exame Época Normal PAW 2020/2021	1
Parte 1 — Escolha Múltipla (7 valores).....	1
Parte 2 — Verdadeiro ou Falso (3 valores).....	5
Parte 3 — Resposta Aberta (10 valores)	5

✓ Resolução — Exame Época Normal PAW 2020/2021

Curso: LEI / LSIRC | **UC:** Programação em Ambiente Web

Duração: 1h30min

Parte 1 — Escolha Múltipla (7 valores)

Cada pergunta vale 1 valor. Opção incorreta desconta 0.5 valores.

1. Indique as afirmações verdadeiras:

- - a. CSS é uma linguagem de programação para a web;
- - b. Numa página da internet escrita em HTML, a informação escrita entre as tags <head>...</head> não é visível na página apresentada no browser;
- - c. Podemos submeter informação para um servidor web sem utilizar JavaScript numa página escrita em HTML;
- - d. O uso de CSS ou JavaScript não é obrigatório numa página HTML.

✓ Respostas corretas: **b, c, d**

Justificação:

- **a) FALSO:** CSS é uma linguagem de *estilos*, não de programação.

- **b) VERDADEIRO:** O <head> contém metadados, links para CSS e scripts, mas não o conteúdo visível (esse fica no <body>). –

c) VERDADEIRO: Podemos usar um formulário HTML padrão (<form action="/rota" method="POST">) que envia dados nativamente sem necessidade de JavaScript. –

d) VERDADEIRO: Uma página pode conter apenas puro HTML (embora fique sem estilos ou interatividade avançada).

2. Entre as seguintes opções escolha os métodos HTTP que podem ser utilizados para enviar dados a um servidor numa API REST:

- - a. GET
- - b. DELETE
- - c. CREATE
- - d. UPDATE

✓ Respostas corretas: **a, b**

Justificação: “GET” e “DELETE” são métodos/verbos HTTP válidos. Embora GET e DELETE não enviem tradicionalmente um *body* (corpo) no pedido, eles enviam dados via **parâmetros de rota** (ex: /user/123) ou **query strings** (ex: ?id=123). As opções “CREATE” e “UPDATE” são operações de base de dados (CRUD), **não** são métodos HTTP (os equivalentes HTTP seriam POST e PUT).

3. Considerando a framework ExpressJS utilizada em Node.js, indique as afirmações verdadeiras:

- - a. Podemos criar componentes com a linguagem de programação TypeScript e o padrão de software MVC;
- - b. Podemos utilizar template engines para gerar páginas HTML dinamicamente no servidor;
- - c. Podemos aceder diretamente a uma base de dados para enviar e guardar informação com o módulo mongoose;
- - d. Podemos executar funções JavaScript da framework ExpressJS diretamente a partir de um browser de internet.

✓ Respostas corretas: **b, c**

Justificação: -

a) FALSO: A criação de “componentes com TypeScript” remete especificamente à framework Angular (frontend). O ExpressJS usa JavaScript padrão. –

b) VERDADEIRO: O Express suporta template engines como o EJS (res.render()). –

c) VERDADEIRO: O Mongoose é o módulo standard para ligar Node.js/Express ao MongoDB. –

d) FALSO: ExpressJS corre no runtime do Node.js (servidor), não no browser.

4. Da seguinte lista selecione as afirmações verdadeiras sobre uma aplicação escrita com a framework Angular:

- - a. Devemos manter as ligações diretas a uma base de dados (ex: MongoDB) para persistir informação da aplicação;
- - b. Podemos utilizar ficheiros de texto para guardar informação da aplicação;
- - c. Podemos utilizar a ferramenta npm para gerir dependências da aplicação;
- - d. Podemos dividir as páginas de internet em componentes reutilizáveis;

 **Respostas corretas: c, d**

Justificação:

- **a) FALSO (Armadilha de segurança):** Nunca se liga o Angular diretamente à base de dados, pois o código fonte fica exposto no browser (cliente).

- **b) FALSO:** Aplicações browser-side não têm acesso direto ao sistema de ficheiros do cliente para guardar “ficheiros de texto” com dados da app. Para persistência usa-se localStorage ou o Backend.

- **c) VERDADEIRO:** O Angular usa o package.json e o npm para gerir bibliotecas.

- **d) VERDADEIRO:** O Angular é baseado numa arquitetura modular de componentes reutilizáveis.

5. Indique as afirmações verdadeiras sobre o desenvolvimento de serviços REST:

- - a. Podemos o mód. swagger para documentar e testar APIs REST;
- - b. Os serviços REST enviam páginas HTML para um cliente;
-

- c. As APIs REST não respeitam a arquitetura cliente-servidor na maioria das aplicações web;
-
- d. É possível utilizar tokens JWT para garantir autenticação e autorização na API REST.

✓ Respostas corretas: a, d

Justificação:

- **b) FALSO:** Serviços REST comunicam através de dados puros (tipicamente formato JSON), não páginas HTML completas. - **c) FALSO:** REST é, por definição, um modelo que obriga estritamente à separação cliente-servidor.

6. Observe a figura 1 e indique as afirmações verdadeiras:

(Excerto de código: `authController.verifyToken = function(req, res, next)... jwt.verify...`) -

a. Está a ser validado um token de autenticação de um utilizador numa aplicação de frontend; - b. Está a ser validado um token de autenticação de um utilizador numa aplicação de backend; - c. A função `verifyToken` não funcionará e resultará num erro 500 sempre que executada; - d. A função `verifyToken` atua como função de middleware na aplicação web. Só no caso de o token ser válido é que o pedido é processado por outras funções;

✓ Respostas corretas: b, d

Justificação: A função recebe `req`, `res`, `next`, sendo inequivocamente um **middleware** da framework ExpressJS (Backend). A função extrai o token e verifica-o. Apenas se passar a verificação é chamado o `next()`, passando o controlo para o próximo middleware/rota.

7. Tendo em consideração a figura 2, indique as afirmações verdadeiras:

(Excerto de código: `@NgModule({ declarations: [AppComponent, PeopleComponent]...})`)

- a. Estamos perante uma aplicação desenvolvida na framework ExpressJS; - b. Estamos perante uma aplicação desenvolvida na framework Angular; - c. A aplicação contém apenas o componente criado por defeito em todas as aplicações e outro criado pelo utilizador; - d. Esta aplicação está a ser desenvolvida para o backend de uma aplicação web.

✓ Respostas corretas: b, c

Justificação: O decorador `@NgModule` e o uso de TypeScript indicam claramente que se trata do ficheiro `app.module.ts` do Angular (frontend). A array `declarations` contém exatamente dois componentes: o `AppComponent` (default) e o `PeopleComponent` (criado pelo utilizador).

Parte 2 — Verdadeiro ou Falso (3 valores)

1. Uma página da internet só pode enviar dados para um servidor backend utilizando o elemento form em HTML com o método POST ou GET.

✗ **FALSO**. O browser pode enviar dados para o servidor de forma assíncrona (sem recarregar a página) usando **AJAX** (via objeto XMLHttpRequest) ou a Fetch API via código JavaScript.

2. JavaScript é uma linguagem de programação que pode ser utilizada no frontend e backend.

✓ **VERDADEIRO**. Corre no browser (frontend) e em servidores (backend) graças a runtimes como o Node.js.

3. JavaScript é a linguagem de programação utilizada em aplicações escritas com a framework Angular.

✗ **FALSO**. A linguagem utilizada no Angular é o **TypeScript** (que é um superset do JavaScript, com tipagem estática e compilação obrigatória).

4. Node.js é uma framework para escrita de aplicações no backend.

✗ **FALSO**. Node.js não é uma framework, é um **runtime** (ambiente de execução) que permite correr JavaScript fora do browser. A framework utilizada sobre o Node.js para aplicações backend é o **ExpressJS**.

5. O padrão de software MVC não pode ser utilizado em aplicações escritas com a framework Angular.

✗ **FALSO**. O Angular usa intrinsecamente o padrão MVC (ou MVVM), onde o “Model” são as classes/serviços com os dados, a “View” é o template HTML, e o “Controller” é a classe TypeScript do componente (*.component.ts).

6. CSS é uma linguagem utilizada para formatar o aspeto de páginas da internet.

✓ **VERDADEIRO**. É a sua função exata (embora não seja uma linguagem de programação).

Parte 3 — Resposta Aberta (10 valores)

1. Indique o que entende pelo conceito cliente-servidor em aplicações web. (1.5 val)

É o modelo de arquitetura base da web composto por dois intervenientes: - O **Cliente** (browser do utilizador) que executa a interface gráfica (frontend com

HTML/CSS/JS) e inicia a comunicação enviando um **pedido HTTP**. - O **Servidor** (computador remoto, ex: Node.js) que recebe o pedido HTTP, executa a lógica de negócio (backend), acede a bases de dados se necessário, e devolve uma **resposta HTTP** ao cliente.

2. Indique as diferentes formas possíveis para incluir CSS numa página de internet escrita em HTML. (1.5 val)

Existem 3 formas de adicionar CSS: 1. **Inline**: Usando o atributo style diretamente na tag HTML (ex: <p style="color:red;">). 2. **Internal**: Colocando código CSS dentro de uma tag <style> na secção <head> do documento HTML. 3. **External**: Referenciando um ficheiro de estilos externo usando a tag <link rel="stylesheet" href="style.css"> na secção <head> (método recomendado).

3. Observe a figura 3. Descreva em que tipo de aplicações podemos encontrar este excerto de código e o seu objetivo. Indique também as funcionalidades esta aplicação aparenta ter e caso seja necessário que correções poderão ser implementadas. (2 val)

(O código mostra um array const routes: Routes = [...]) - **Tipo de Aplicação**: Aplicação Frontend do tipo SPA (Single Page Application) desenvolvida em **Angular**. –

Objetivo: Configurar o **Routing** (Navegação) da aplicação, mapeando URLs específicos para componentes Angular respetivos sem recarregar a página. –

Funcionalidades: A aplicação possui páginas de “login”, “register”, um “profile” com ID dinâmico, e “aboutus”. Utiliza também um **Guard** (AuthGuardService) para tentar proteger o acesso a rotas.

Correções necessárias:

1. Falta uma vírgula , após component: LoginComponent.
2. Falta a aspa inicial nas strings das rotas register e profile/:id (estão como path: register' em vez de path: 'register').
3. **Erro lógico**: As rotas de login e register NÃO devem estar protegidas com o canActivate: [AuthGuardService], pois isso impediria utilizadores não-autenticados de fazerem login/registo! O Guard só deve proteger a rota do profile.

4. Observe a figura 4 e descreva de forma sucinta a funcionalidade implementada na página HTML. Caso identifique erros na página indique como podem ser resolvidos. (2 val)

- **Funcionalidade**: A página pretende obter dois números a partir de inputs do utilizador, multiplicá-los quando se clica num botão e exibir o resultado numa div inferior através de JavaScript DOM manipulation.
- **Erros identificados e resoluções**:

1. O segundo input usa o atributo `class="arg2"`, mas o JavaScript tenta selecioná-lo usando `document.getElementById("arg2")`. **Resolução:** Alterar para `id="arg2"` no HTML.
2. O script tenta adicionar o evento de clique num elemento com ID "calcular" (`document.getElementById("calcular")`), mas o botão não tem nenhum ID atribuído. **Resolução:** Adicionar `id="calcular"` na tag `<button>`.

5. Onde e de que forma devemos validar a informação submetida por utilizadores em aplicações web. (3 val)

A validação de dados deve ser **OBRIGATORIAMENTE** implementada em ambos os lados da aplicação:

- **No Frontend (Cliente):** Serve para melhorar a Experiência do Utilizador (UX), dando feedback imediato. Pode ser feito usando validações nativas de HTML5 (atributos como `required`, `minlength`, `type="email"`) ou via validação reativa do Angular.

- **No Backend (Servidor):** Serve para garantir a **Segurança** do sistema. A validação frontend pode ser facilmente contornada (ex: desativando JS ou fazendo pedidos diretos via API/Postman). O servidor deve **SEMPRE** validar e sanitizar toda a informação recebida antes de interagir com a base de dados (ex: usando middlewares no ExpressJS ou validações do schema do Mongoose) para evitar injeção de código ou dados corrompidos.